

Organização da Implementação em Java

Autômato Celular Baseado em Populações

Objetivo

Organizar a implementação de um autômato celular em Java no qual cada célula representa uma construção e a população é representada por parâmetros de qualidade de vida, felicidade e crescimento.

1. Estrutura do projeto

Uma estrutura simples, adequada para um projeto acadêmico:

```
src/
├── main/
│   └── java/
│       ├── Main.java
│       └── automato/
│           ├── Celula.java
│           ├── EstadoCelula.java
│           ├── Grade.java
│           ├── Populacao.java
│           ├── Regras.java
│           └── Simulacao.java
```

2. Responsabilidade de cada classe

Classe	Responsabilidade
Main	Inicializar o programa, receber parâmetros e iniciar a simulação.
Celula	Representar uma construção e guardar posição e estado.
EstadoCelula	Enum com os estados possíveis da construção.
Grade	Manter a matriz de células e fornecer acesso aos vizinhos.
Populacao	Manter qualidade de vida, felicidade e dados de crescimento.
Regras	Concentrar as regras que determinam a evolução do sistema.
Simulacao	Controlar os ciclos e aplicar as alterações da grade.

3. EstadoCelula.java

Usar um enum para evitar números soltos representando estados:

```
public enum EstadoCelula {
    ABANDONADA,
    BAIXA_QUALIDADE,
    MEDIA_QUALIDADE,
    ALTA_QUALIDADE
}
```

4. Celula.java

A célula deve ser simples. Campos sugeridos:

```
private int x;  
private int y;  
private EstadoCelula estado;
```

Métodos principais:

```
getX(), getY(), getEstado(), setEstado()
```

5. Grade.java

Responsável pela matriz de células e pela vizinhança.

```
private Celula[][] celulas;  
private int largura;  
private int altura;
```

Métodos sugeridos:

```
obterCelula(x, y)  
obterVizinhos(x, y)  
inicializar()
```

Usar inicialmente a vizinhança de Moore, considerando as oito posições ao redor da célula.

6. Populacao.java

Representa os parâmetros populacionais usados pelas regras do autômato.

```
private int qualidadeDeVida; // 0, 1 ou 2
private double felicidade; // 0 a 100
private int populacao;
private double taxaDeCrescimento;
```

7. Regras.java

Concentrar aqui as regras matemáticas e lógicas do modelo. A classe não deve controlar a simulação inteira.

```
calcularFelicidade(Populacao populacao)
calcularTaxaCrescimento(Populacao populacao)
determinarQualidadeNovaCelula(Populacao populacao)
calcularProximoEstado(Celula celula, List<Celula> vizinhos, Populacao populacao)
```

8. Regras iniciais do modelo

Qualidade de vida: definida pelo usuário e limitada a 0, 1 ou 2.

```
QV = 0 → felicidade diminui gradualmente
QV = 1 → felicidade permanece aproximadamente estável
QV = 2 → felicidade aumenta gradualmente
```

A felicidade deve ser limitada ao intervalo 0–100.

9. Crescimento

Uma primeira implementação pode utilizar a felicidade como base da probabilidade de crescimento:

```
double chance = populacao.getFelicidade() / 100.0;
boolean cresce = Math.random() < chance;
```

Depois, a fórmula pode ser refinada para incluir vizinhança, espaço disponível e outros fatores.

10. Qualidade das novas construções

A qualidade da construção criada é determinada inicialmente pela qualidade de vida:

```
QV 0 → ABANDONADA ou BAIXA_QUALIDADE
QV 1 → MEDIA_QUALIDADE
QV 2 → ALTA_QUALIDADE
```

11. Simulacao.java

Controla a passagem do tempo e deve trabalhar com dois estados da grade: o estado atual e o próximo estado.

```
executarCiclo():
1. Atualizar população.
2. Percorrer todas as células.
3. Consultar os vizinhos.
4. Calcular o próximo estado.
5. Guardar o próximo estado sem alterar a grade atual.
6. Ao terminar o percurso, aplicar todos os estados novos.
7. Avançar o contador de ciclos.
```

12. Por que usar o próximo estado?

Não modificar a célula imediatamente durante o percurso da matriz. Caso contrário, uma célula analisada depois poderia enxergar uma alteração feita por uma célula analisada antes no mesmo ciclo. Isso quebra a ideia de uma atualização simultânea típica de autômatos celulares.

13. Fluxo geral

```
Main
↓
cria Populacao
↓
cria Grade
↓
cria Simulacao
↓
executa ciclo
↓
Populacao é atualizada
↓
Grade é analisada
↓
Regras calculam próximos estados
↓
novos estados são aplicados
↓
próximo ciclo
```

14. Ordem recomendada de implementação

1. Criar o enum EstadoCelula.
2. Criar Celula e seus getters/setters.
3. Criar Grade e testar a matriz.
4. Implementar obterVizinhos().
5. Criar Populacao.
6. Implementar atualização da felicidade.
7. Implementar cálculo de crescimento.
8. Criar Regras e implementar as transições básicas.
9. Criar Simulacao e implementar ciclos.
10. Criar Main para configurar e executar o modelo.
11. Testar diferentes valores de qualidade de vida.
12. Somente depois adicionar regras mais complexas.

15. Primeira versão funcional

Para validar o conceito, a primeira versão deve ter somente: uma grade, quatro estados de construção, qualidade de vida 0–2, felicidade 0–100, crescimento probabilístico e influência dos vizinhos. Depois que essa versão estiver funcionando, podem ser adicionados fatores como população por célula, economia, emprego, recursos ou diferentes tipos de construção.

16. Checklist antes de considerar a base pronta

- Grade é criada corretamente
- Estados das células podem mudar
- Vizinhos são encontrados corretamente
- QV é configurável pelo usuário
- Felicidade muda gradualmente
- Crescimento depende da felicidade
- Qualidade das novas construções depende da QV
- Atualização ocorre simultaneamente
- Simulação executa vários ciclos sem erro